

Biometric-Bound Post-Quantum Encrypted File Transmission

with Forensic Traitor Tracing

Hardware Connection Setup & Project Run Guide

1. Hardware Components Overview

The system requires the following hardware components to be assembled before running any software. Bob's laptop hosts all biometric hardware; Alice's laptop requires only WiFi connectivity.

Component	Specification	Used By	Purpose
Laptop (Alice)	Windows 11, Python 3.11, WiFi	Alice	Sender — encrypts & transmits files
Laptop (Bob)	Windows 11, Python 3.11, WiFi	Bob	Receiver — authenticates, decrypts, watermarks
R307 Fingerprint Sensor	Optical, UART, 3.3V/5V	Bob	Biometric authentication & enrollment
CP2102 USB-to-TTL Converter	USB-A, 3.3V/5V selectable	Bob	Bridges R307 serial to laptop USB port
Jumper Wires (4x)	Female-to-Female recommended	Bob	Connects R307 to CP2102 module
USB Cable (Type-A to Mini/Micro)	Standard USB	Bob	Connects CP2102 module to Bob's laptop

2. Hardware Connection Setup (Bob's System)

The R307 fingerprint sensor communicates over UART serial at 57600 baud. It cannot be plugged directly into a laptop USB port — the CP2102 USB-to-TTL converter is the bridge. Follow the steps below carefully.

2.1 R307 Sensor Pin Identification

The R307 sensor has a 6-wire ribbon cable. Only 4 wires are needed. The wire colors may vary by manufacturer — always verify with your sensor's datasheet, but the standard wiring is:

R307 Wire Color (Typical)	Pin Name	Function	Connect To (CP2102)
Red	VCC (Power)	3.3V or 5V supply	VCC (3.3V or 5V pin on CP2102)
Black	GND (Ground)	Common ground	GND pin on CP2102
Yellow / White	TXD (Transmit)	Sensor sends data to laptop	RXD pin on CP2102
Green / Blue	RXD (Receive)	Sensor receives commands	TXD pin on CP2102
Brown / Purple	3.3V Wakeup (optional)	Touch sensing wakeup	Leave unconnected
Orange (optional)	NC or IRQ	Not used in this project	Leave unconnected

CRITICAL: TX of R307 connects to RX of CP2102, and RX of R307 connects to TX of CP2102. Cross the TX/RX lines — do NOT connect TX-to-TX.

CRITICAL: Set the CP2102 voltage jumper to 3.3V if your R307 is rated 3.3V. Most R307 modules accept 3.3V-5V but check your specific unit to avoid damage.

2.2 Step-by-Step Wiring Instructions

1. Place both the R307 sensor and the CP2102 module on a flat, non-conductive surface.
2. Identify the 4 wires from the R307 ribbon: VCC (Red), GND (Black), TXD (Yellow/White), RXD (Green/Blue).
3. Using jumper wires, connect each R307 wire to the corresponding CP2102 pin as listed in the table above (Section 2.1).
4. Set the voltage selection jumper on the CP2102 board to 3.3V (or 5V if your R307 requires it).
5. Plug the CP2102 module into Bob's laptop via the USB cable.
6. Windows 11 will auto-install the CP2102 driver. Wait 10-15 seconds for the device to initialize.
7. Open Device Manager (Win + X > Device Manager) and expand 'Ports (COM & LPT)'. Note the COM port assigned — e.g., COM3, COM4, COM5.
8. Update the COM port number in all Bob's Python scripts where the serial port is referenced (typically in enroll.py, authenticate.py, and biometric/ scripts).

2.3 Verifying the Hardware Connection

Before running any project scripts, verify the R307 sensor responds correctly:

```
cd Bob_System/biometric
python test_r307.py
```

A successful output looks like:

```
[INFO] Opening serial port COM4 at 57600 baud...  
[INFO] Sending handshake to R307...  
[SUCCESS] R307 responded. Sensor online.
```

If you see: 'serial.SerialException: could not open port' — the COM port number in the script is wrong. Re-check Device Manager and update the port.

If you see: 'No response from sensor' — check TX/RX are not swapped, and verify power connections.

3. Software Prerequisites

3.1 Python Packages (Both Systems)

Install the required Python libraries on both Alice's and Bob's laptops. Open a terminal (cmd or PowerShell) and run:

```
pip install pyserial cryptography
```

Bob's system additionally requires the SQLite database module, which is bundled with Python 3.11 by default.

3.2 COM Port Configuration (Bob Only)

Open the following files in Bob_System and ensure the COM port matches what Device Manager shows:

File	Variable to Edit	Example Value
biometric/test_r307.py	SERIAL_PORT	COM4
enroll.py	SERIAL_PORT	COM4
authenticate.py	SERIAL_PORT	COM4
biometric/clear_sensor.py	SERIAL_PORT	COM4
biometric/extract_template.py	SERIAL_PORT	COM4

Tip: COM port numbers can change each time you re-plug the USB cable. Always verify in Device Manager before running any script.

3.3 Network Configuration

Alice and Bob must be on the same WiFi network. Before running the project, find Bob's IP address:

```
ipconfig      (on Bob's laptop in cmd)
```

Look for the IPv4 address under the WiFi adapter — e.g., 192.168.1.105. Update this IP in Alice's scripts:

File (Alice_System)	Variable / Parameter	Set To
get_public_key.py	BOB_IP or HOST	Bob's WiFi IPv4 address
send_file.py	BOB_IP or HOST	Bob's WiFi IPv4 address

Firewall: Windows Firewall may block incoming connections on Bob's laptop. If connections fail, temporarily disable the firewall for the Private network profile, or add inbound rules for TCP ports 5000 and 6000.

4. File Structure Reference

The project is organized into two independent system folders — one per laptop. The complete structure is shown below:

Alice_System (Sender Laptop)

File / Folder	Role
encrypt.py	Encrypts the plaintext file using AES-256-GCM and encapsulates the AES key with Bob's public key (XOR-based KEM)
get_public_key.py	Connects to Bob's public key registration server (port 5000) and downloads bob_public.key
send_file.py	Transmits the encrypted package (encrypted.bin) to Bob over TCP port 6000
extract_watermark.py	Extracts and decodes the invisible zero-width Unicode watermark from a received watermarked document
message.txt	The plaintext input file Alice wants to send securely to Bob
bob_public.key	Bob's Kyber-derived public key (downloaded during Phase 2 — Public Key Registration)
encrypted.bin	Output of encrypt.py — the complete encrypted package: [key_len enc_key nonce ciphertext]

File / Folder	Role
decrypted_watermarked.txt	If Alice receives the watermarked file back, she can run extract_watermark.py on this
__pycache__ /	Python bytecode cache — auto-generated, safe to ignore

Bob_System (Receiver Laptop)

File / Folder	Role
enroll.py	Runs once to register Bob's fingerprint with the system. Generates master secret, stores helper data in SQLite
public_key_registration.py	Starts TCP server on port 5000. Requires Bob's fingerprint, regenerates keypair in RAM, sends public key to Alice
file_receiver.py	Listens on TCP port 6000 for incoming encrypted packages from Alice. Saves as received_package.bin
decrypt_and_watermark.py	Requires fingerprint, regenerates private key in RAM, decrypts received_package.bin, embeds invisible watermark, saves decrypted_watermarked.txt
authenticate.py	Standalone biometric test/authentication module — used internally by other scripts
test_kyber.py	Unit test for the PQC key generation (keygen.py) — verifies key derivation is deterministic
received_package.bin	Encrypted file package received from Alice over the network
decrypted_watermarked.txt	Final output: the decrypted document with invisible watermark embedded
biometric/	Sub-module for R307 sensor interaction
biometric/test_r307.py	Hardware test — verifies R307 is connected and responding
biometric/clear_sensor.py	Clears all fingerprint templates stored in R307's internal memory
biometric/extract_template.py	Extracts template position from sensor for seed derivation
biometric/feature_reduce.py	Dimensionality reduction on biometric features (for fuzzy extractor helper data)
biometric/quantity.py	Checks how many template slots are used on the R307 sensor
database/db.py	SQLite helper module — handles reading/writing user records (uid, template_position, master_secret)
database/users.db	SQLite database file — stores enrollment data (helper data only, no raw fingerprint)
pqc/keygen.py	Core PQC module — SHAKE-256 based deterministic key generation from biometric seed
pqc/__init__.py	Python package initializer for the pqc module

5. Running the Project — Step-by-Step

The project runs across 5 sequential phases. Execute them in the exact order shown below. Both terminals (Alice and Bob) must remain open throughout the session.

Phase 1: Biometric Enrollment (Bob — One-Time Setup)

This phase only needs to be run once. It registers Bob's fingerprint and stores the helper data (master secret + template position) in the local SQLite database. The private key is never saved — it is only generated transiently in RAM during enrollment to extract the public key.

On Bob's laptop, open a terminal in Bob_System:

```
cd Bob_System
python enroll.py
```

When prompted:

- Place your finger firmly on the R307 sensor surface.
- Hold for 2-3 seconds until the LED blinks or terminal confirms capture.
- Remove and re-place your finger a second time when prompted (consistency check).
- Enrollment succeeds when the terminal prints: [SUCCESS] Enrollment complete. Public key saved.

Important: Ensure the fingerprint is centered on the sensor lens for the best template quality. Poor placement causes false rejections during decryption later.

Reset: To re-enroll (e.g., after changing hardware), first run `python biometric/clear_sensor.py` to wipe the R307 template memory, then delete `users.db` from the `database/` folder, then run `enroll.py` again.

Phase 2: Public Key Registration (Bob Starts Server, Alice Connects)

Bob starts a TCP server that will authenticate him via fingerprint and transmit his public key to Alice. Both laptops must be on the same WiFi network.

Step 1 — On Bob's laptop (terminal in Bob_System):

```
python public_key_registration.py
[INFO] Public key server started on port 5000. Waiting for Alice...
```

Step 2 — On Alice's laptop (terminal in Alice_System):

```
cd Alice_System
```

```
python get_public_key.py
```

What happens next:

9. Alice's script connects to Bob's server at the configured IP and port 5000.
10. Bob's terminal prompts: 'Place finger to authenticate and release public key.'
11. Bob places his enrolled finger on the R307 sensor.
12. The system regenerates the seed and keypair in RAM, transmits only the public key to Alice, then deletes the private key from RAM.
13. Alice's script saves the received key as bob_public.key in Alice_System.
14. Both terminals confirm: [SUCCESS] Public key transfer complete.

Verify: Check that bob_public.key now exists in the Alice_System folder and is non-zero in size before proceeding.

Phase 3: File Encryption and Transmission (Alice Sends to Bob)

Alice encrypts the confidential file (message.txt) and transmits it to Bob. Bob must have the file receiver running before Alice sends.

Step 1 — On Bob's laptop (new terminal or same one, in Bob_System):

```
python file_receiver.py  
[INFO] File receiver listening on port 6000...
```

Step 2 — On Alice's laptop, first encrypt the file:

```
python encrypt.py
```

encrypt.py performs:

- Reads message.txt (the plaintext document)
- Generates a random 256-bit AES-256-GCM session key KAES
- Encrypts the file: Cfile = AES-256-GCM(KAES, plaintext, nonce)
- Encapsulates the key: Ckey = KAES XOR PK_Bob[:32]
- Assembles the package: PKG = [len(Ckey) | Ckey | nonce | Cfile]
- Saves the result as encrypted.bin

Step 3 — Transmit the encrypted package to Bob:

```
python send_file.py  
[INFO] Connecting to Bob at 192.168.x.x:6000...  
[SUCCESS] Encrypted package transmitted.
```

Bob's file_receiver.py terminal confirms receipt and saves received_package.bin.

Phase 4: Biometric Decryption and Watermark Embedding (Bob)

Bob authenticates with his fingerprint to decrypt the received package. Immediately after decryption, an invisible forensic watermark is embedded into the document before it is written to disk.

On Bob's laptop (terminal in Bob_System):

```
python decrypt_and_watermark.py
```

Process flow:

15. Script loads received_package.bin and parses [Ckey | nonce | Cfile].
16. Terminal prompts: 'Place enrolled finger to authenticate.'
17. Bob places finger on R307 sensor. System reconstructs seed, regenerates private key in RAM.
18. AES key recovered: KAES = Ckey XOR PK_Bob[:32].
19. Plaintext recovered: P = AES-256-GCM-Dec(KAES, Cfile, nonce).
20. Watermark generated: sigma = SHA-256(uid || master_secret).
21. Payload assembled: payload = uid || timestamp || sigma.
22. Payload encoded as zero-width Unicode characters (U+200B for 0, U+200C for 1) and appended to the decrypted text.
23. Final watermarked document saved as decrypted_watermarked.txt.
24. Private key deleted from RAM.

Terminal output on success:

```
[SUCCESS] Fingerprint authenticated.  
[SUCCESS] AES key recovered. Decryption complete.  
[SUCCESS] Watermark embedded. File saved: decrypted_watermarked.txt
```

Phase 5: Forensic Watermark Extraction (Investigation Scenario)

If a leaked copy of the document is found, the watermark extractor identifies the responsible user. This module can be run on either laptop. Provide the leaked document as input.

On Alice's laptop (or any investigation machine) with the leaked file:

```
python extract_watermark.py
```

The script:

- Scans the document for U+200B and U+200C zero-width Unicode characters.
- Reconstructs the binary string from the character sequence.
- Decodes ASCII payload: uid, timestamp, SHA-256 signature.
- Prints forensic evidence to the terminal.

Example forensic output:

```
=== FORENSIC WATERMARK EXTRACTED ===
User ID      : bob_user_001
Timestamp    : 2025-04-08 14:32:17
Signature     : a3f7c2...d91e (SHA-256)
=====
```

Note: The zero-width watermark is preserved when the document is copied, forwarded via email, or uploaded to cloud services — since it is part of the text content. It does NOT survive screenshots or re-typed content.

6. Quick Run Order Reference

Phase	Who	Command	Port
1 — Enrollment (once)	Bob	python enroll.py	—
2a — Start Key Server	Bob	python public_key_registration.py	5000
2b — Download Public Key	Alice	python get_public_key.py	5000
3a — Start File Receiver	Bob	python file_receiver.py	6000
3b — Encrypt File	Alice	python encrypt.py	—
3c — Send Encrypted File	Alice	python send_file.py	6000
4 — Decrypt + Watermark	Bob	python decrypt_and_watermark.py	—
5 — Extract Watermark	Alice / Bob	python extract_watermark.py	—

7. Troubleshooting Common Issues

Symptom	Likely Cause	Fix
serial.SerialException: could not open port	Wrong COM port in script	Open Device Manager > Ports. Update SERIAL_PORT in all biometric scripts to the correct COM number (e.g., COM4).

Symptom	Likely Cause	Fix
No response from R307 sensor	TX/RX swapped OR power issue	Verify: R307-TXD goes to CP2102-RXD and vice versa. Check power voltage matches sensor spec.
Fingerprint authentication fails consistently	Poor enrollment template	Run clear_sensor.py, delete users.db, re-enroll with firm centered finger placement.
ConnectionRefusedError on Alice	Bob's server not running or firewall blocking	Start Bob's server first. Disable Windows Firewall for Private networks or add TCP inbound rules for ports 5000 and 6000.
bob_public.key not found in Alice_System	Phase 2 did not complete	Re-run public_key_registration.py on Bob and get_public_key.py on Alice. Verify both print SUCCESS.
Decryption fails / garbage output	Mismatched key (wrong fingerprint or corrupt DB)	Ensure only the enrolled user's finger is used. Do not modify users.db manually.
Watermark not detected by extract_watermark.py	File was screenshot or retyped	Zero-width watermarks only survive digital copy-paste. Screenshots and retyping strip the watermark.
CP2102 not appearing in Device Manager	Driver not installed	Download CP2102 driver from Silicon Labs (silabs.com) and install manually.